



PROGRAMMING WITH PYTHON

Lekha A

Computer Applications

PROGRAMMING WITH PYTHON

Exception Handling, File I/O, Modules and Packages

File I/O Operations - XML

Lekha A

Computer Applications



PROGRAMMING WITH PYTHON

XML Files

- The Extensible Markup Language (XML) is a markup language much like HTML.
- It is a portable.
- It is useful for handling small to medium amounts of data without using any SQL database.



PROGRAMMING WITH PYTHON

XML Files

- Python's standard library contains xml package.
- This package has following modules that define XML processing APIs.
 - `xml.etree.ElementTree`: a simple and lightweight XML processor API
 - `xml.dom`: the DOM API definition
 - `xml.sax`: SAX2 base classes and convenience functions



PROGRAMMING WITH PYTHON

XML Files - ElementTree module

- XML is a tree like hierarchical data format.
 - This module has two classes for this purpose
 - 'ElementTree' treats the whole XML document as a tree
 - 'Element' represents a single node in this tree.



PROGRAMMING WITH PYTHON

XML Files - ElementTree module

```
import xml.etree.ElementTree as xml
```

Import the package

```
books = [  
    {'title': 'Pride and Prejudice', 'author': 'Jane Asuten',  
     'price': 240, 'category': 'fiction'},  
    {'title': 'The Diary of a Young Girl', 'author': 'Anne Frank',  
     'price': 560, 'category': 'non-fiction'}  
]
```

```
root=xml.Element("books")
```

List of books with attributes and elements

Create the root element



PROGRAMMING WITH PYTHON

XML Files - ElementTree module

```
for book in books:
    book_element = xml.Element("book")
    root.append(book_element)
    book_element.set("category", book['category'])
    title = xml.SubElement(book_element, "title")
    title.text = book['title']
    author = xml.SubElement(book_element, "author")
    author.text = book['author']
    price = xml.SubElement(book_element, "price")
    price.text = str(book['price'])
tree = xml.ElementTree(root)
with open('books.xml', "wb") as fh:
    tree.write(fh)
print("books.xml file created successfully!")
```

Iterate through the books list to build the XML tree

```
▼<books>
  ▼<book category="fiction">
    <title>Pride and Prejudice</title>
    <author>Jane Asuten</author>
    <price>240</price>
  </book>
  ▼<book category="non-fiction">
    <title>The Diary of a Young Girl</title>
    <author>Anne Frank</author>
    <price>560</price>
  </book>
</books>
```



PROGRAMMING WITH PYTHON

XML Files – Tree Structure

- Root Element
 - The topmost element of the XML document.
 - Every XML document has exactly one root element.
 - `<bookstore>`: bookstore is the root.



PROGRAMMING WITH PYTHON

XML Files - Tree Structure

- Parent Element
 - An element that contains one or more child elements.
 - In `<book><title>Pride and Prejudice</title></book>`, book is the parent of title.



PROGRAMMING WITH PYTHON

XML Files - Tree Structure

- Child Element
 - An element that is nested inside a parent element.
 - In `<book><title>Pride and Prejudice</title></book>`, title is the parent of title.



PROGRAMMING WITH PYTHON

XML Files - Tree Structure

- Sibling Elements
 - Elements that share the same parent.
 - In `<book><title>Pride and Prejudice</title><author>Jane Austen</author></book>`, title and author are siblings



PROGRAMMING WITH PYTHON

XML Files - Tree Structure

- Attributes
 - Name-value pairs associated with an element.
 - Attributes are not considered nodes but are part of the element.
 - `<book category="fiction">` where `category="fiction"` is an attribute



PROGRAMMING WITH PYTHON

XML Files – XML Files – ElementTree Module

- Parsing an XML File

```
import xml.etree.ElementTree as xml
```

```
tree = xml.ElementTree(file='books.xml')  
root = tree.getroot()  
books = []
```

Import the package

Parse the XML file and initialize
an empty list to store book
details



PROGRAMMING WITH PYTHON

XML Files – XML Files – ElementTree Module

- Parsing an XML File

```
for book in root.findall('book'):
    book_data = {}
    book_data['category'] = book.get('category')
    for data in book:
        book_data[data.tag] = data.text
    books.append(book_data)
print(books)
```

Iterate over each 'book' element in the XML, Extract the category attribute, Extract the child elements: title, author and price, Append the extracted book details to the books list and Print the parsed books data



PROGRAMMING WITH PYTHON

XML Files – XML Files – ElementTree Module

```
[{'category': 'fiction', 'title': 'Pride and Prejudice', 'author': 'Jane Asuten', 'price': '240'}, {'category': 'non-fiction', 'title': 'The Diary of a Young Girl', 'author': 'Anne Frank', 'price': '560'}]
```



PROGRAMMING WITH PYTHON

XML Files – XML Files – ElementTree Module

- Modifying an XML File

```
import xml.etree.ElementTree as xml
```

```
tree = xml.ElementTree(file='myfile.xml')  
root = tree.getroot()
```

```
for price_element in root.iter('price'):  
    price = int(price_element.text)  
    price += 10  
    price_element.text = str(price)
```

Iterate over each 'price' element in the XML, Convert the price to an integer, increment by 10, and update the text



PROGRAMMING WITH PYTHON

XML Files – XML Files – ElementTree Module

- Modifying an XML File

```
with open("myfile.xml", "wb") as fh:  
    tree.write(fh)  
print("Prices updated successfully!")
```

Save the modified XML back to
the file

```
▼<books>  
  ▼<book category="fiction">  
    <title>Pride and Prejudice</title>  
    <author>Jane Asuten</author>  
    <price>240</price>  
  </book>  
  ▼<book category="non-fiction">  
    <title>The Diary of a Young Girl</title>  
    <author>Anne Frank</author>  
    <price>560</price>  
  </book>  
</books>
```



PROGRAMMING WITH PYTHON

Recap

- XML Files
- File Handling - Element Tree
- Operations
 - Reading
 - Writing
 - Modifying



THANK YOU

Lekha A

Computer Applications